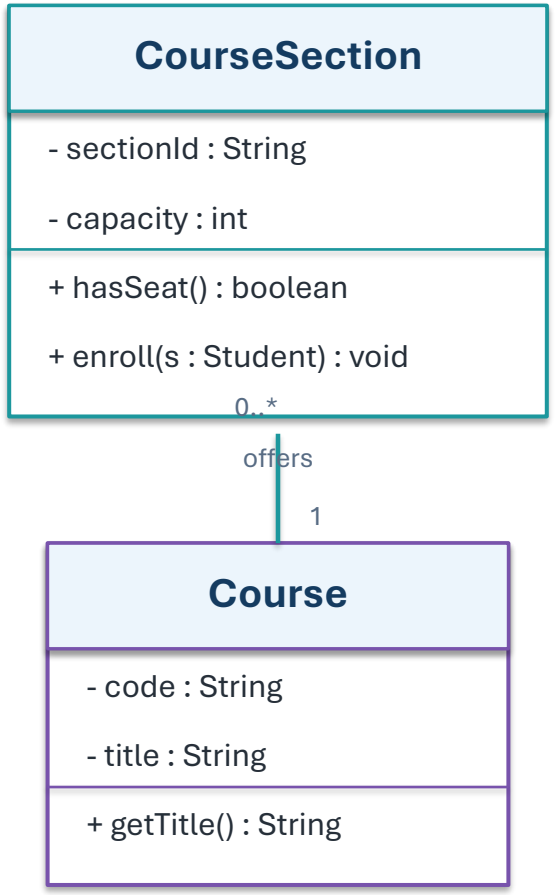
LECTURE 11

# UML Class Diagrams

Turning classes, responsibilities, and relationships into a precise visual model.

Continuation: composition & class design → formal UML notation




Muhammad Shahid Bhatti
Assistant Professor • Department of Computer Science • Office H-17

Course Repository: github.com/mshahidbhatti/CSC241-OOP

*Reference: CSC241 Course Handbook • Liang, Ch. 8*


© Muhammad Shahid Bhatti

CONTINUITY

# Bridge from Lecture 10

Lecture 10 gave us relationship semantics. Lecture 11 gives those relationships a standard visual vocabulary.

## Lecture 10 — design meaning

**Composition, aggregation, association**

Ownership and lifetime dependency

Cohesion and coupling

Nested object graphs

Composition vs. inheritance

## Lecture 11 — visual notation

**Class boxes and compartments**

Visibility + attributes + operations

Multiplicity and relationship symbols

Generalization and dependency

Abstract classes/operations

## Key shift

**From "we understand the design"**

to

**"we can communicate the design precisely."**

A diagram is useful only when another developer can read it consistently.


© Muhammad Shahid Bhatti

CDF ALIGNMENT

# Lecture 11 scope from the course handbook

We stay focused on class-diagram notation; forward/reverse engineering comes in Lecture 12.

### 1 Class diagram notation

Class boxes, compartments, names and basic conventions.

### 2 Attributes / operations / visibility

Represent data and behavior with UML syntax.

### 3 Relationships + multiplicity

Association, aggregation, composition and cardinality.

### 4 Generalization + dependency

Inheritance notation and temporary uses-a relationships.

### 5 Abstract classes / operations

Show incomplete base abstractions correctly.

*Next lecture: systematic Java ↔ UML mapping and tooling.*


© Muhammad Shahid Bhatti

# Learning outcomes

By the end of the lecture, students should be able to read and draw a useful class diagram—not just recognize symbols.

### Read

Interpret a class box, visibility, types, operations and multiplicity.

### Draw

Represent classes and relationships using correct UML symbols.

### Distinguish

Association, aggregation, composition, generalization and dependency.

### Explain

Why a multiplicity or relationship choice matches the domain.

### Model

Represent an abstract superclass and abstract operation.

### Validate

Check whether a diagram tells the same story as the requirements.


© Muhammad Shahid Bhatti

MOTIVATION

# Why draw a UML class diagram?

Code is executable; a class diagram is a compact map of structure and relationships.

## Without a shared notation

Boxes and arrows can mean different things to different people.

Relationship strength is ambiguous.

Cardinality is hidden in prose.

Review conversations become slower.

## With UML class notation

A consistent class vocabulary.

Visible responsibilities and public operations.

Explicit multiplicity.

Relationship semantics become inspectable.

## But remember

A diagram is a model, not the program.

Do not add every private implementation detail.

Use the level of detail appropriate to the discussion.

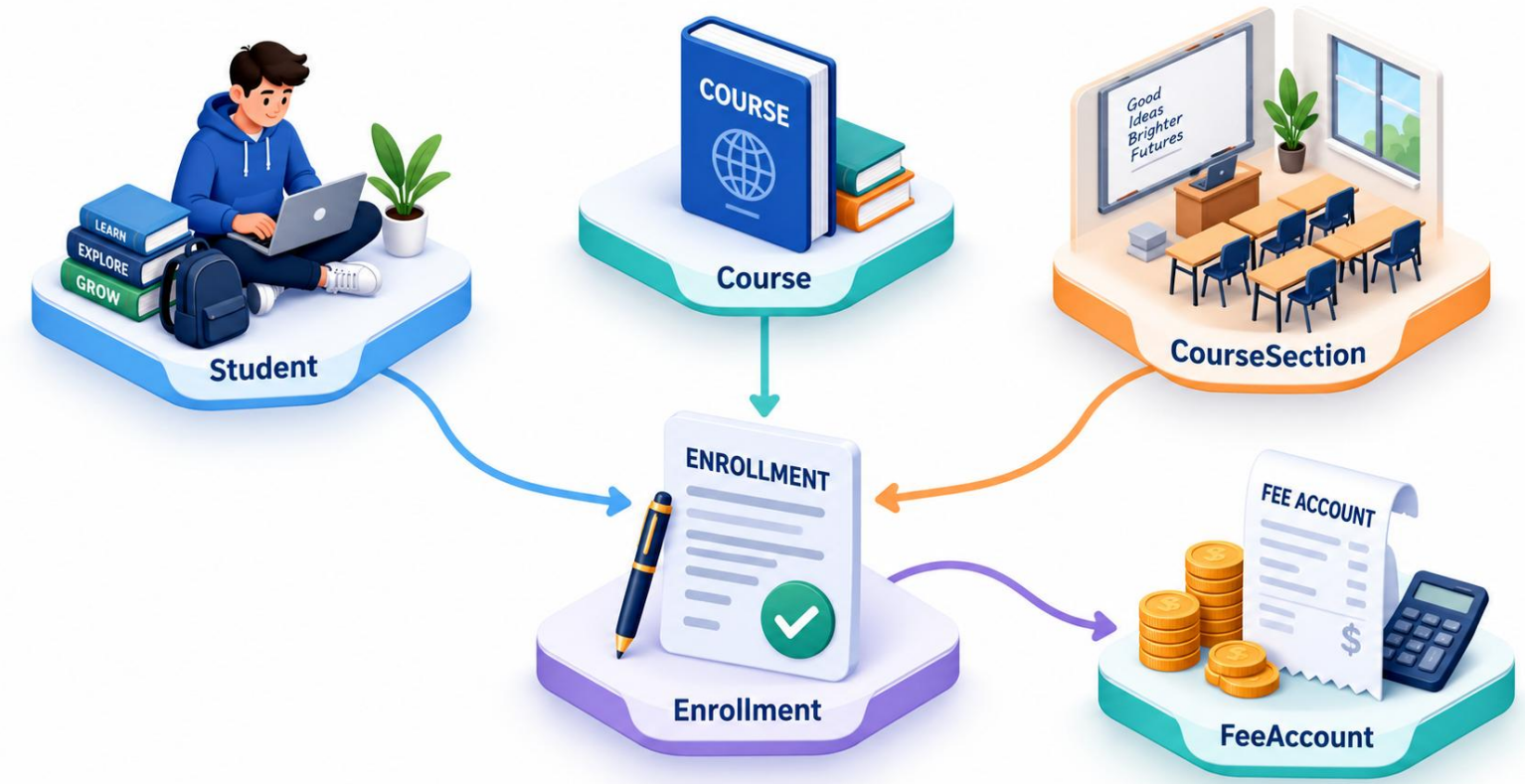
**Best classroom habit: draw just enough structure to explain and challenge the design.**


© Muhammad Shahid Bhatti

VISUAL BRIDGE

# From domain concepts to a class-level model

The same university concepts from Lectures 8–10 can now be expressed with precise UML relationships.



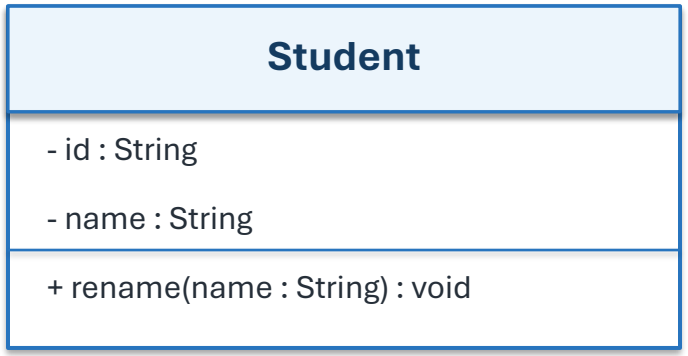

First see the domain concepts. Next, replace pictures with precise class notation.


© Muhammad Shahid Bhatti

CLASS LEVEL

# Class diagram ≠ object diagram

Lecture 11 models types and relationships. Runtime instances belong to a different view.




Class-level statement:
“Every Student has these features.”

**s1 : Student**

id = "SP26-BCS-001"
name = "Abdul Ur Rafe"

**s2 : Student**

id = "SP26-BAI-005"
name = "Abeer Amina"

**Rule of thumb**

Class diagram → types, structure, relationships.

Object diagram → specific instances and links at one moment.

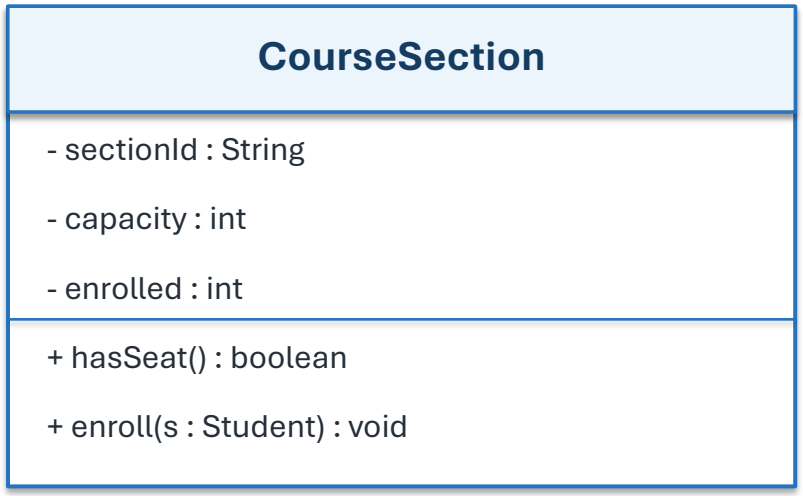
Today we stay at class level.


© Muhammad Shahid Bhatti

NOTATION

# Anatomy of a UML class box

A class is usually shown with three compartments: name, attributes, and operations.




### 1 Class name

Type or abstraction being modeled.

### 2 Attributes

Important state, usually typed.

### 3 Operations

Behavior offered by the class.

### Do we always show everything?

No.

A conceptual diagram may show only important attributes/operations.

A design-level diagram may include more detail.

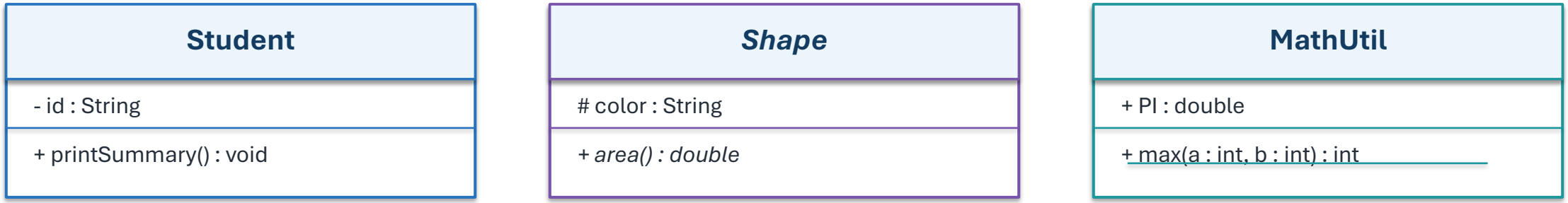
UML is selective abstraction, not a source-code dump.


© Muhammad Shahid Bhatti

CLASS NAME

# Name, abstract, and static conventions

UML uses typography as part of the notation.




Normal class name
usually centered + bold

Abstract class / operation
commonly italicized

Static members are
traditionally
shown underlined.


© Muhammad Shahid Bhatti

# Visibility symbols

UML compresses access level into one character before a member name.

**+ public**

Accessible wherever the language permits public access.

**- private**

Hidden inside the class; most fields should often be private.

**# protected**

Visible to the class and, conceptually, its specialization context.

**~ package / default**

Package-level visibility in Java.

Example: - balance : double + deposit(amount : double) : void


© Muhammad Shahid Bhatti

# Attribute notation

A compact attribute declaration communicates visibility, name, type, and optionally a default value.

### General form

visibility  name : Type  = defaultValue

- id : String

- capacity : int = 40

# status : Status

+ MAX_SIZE : int = 100

### Read the notation

The colon separates the attribute name from its type.

The type is part of the model, not an implementation comment.

Default values are optional.

Collection implementation details can often be replaced by multiplicity at conceptual level.

Keep the diagram readable—do not list every temporary variable.


© Muhammad Shahid Bhatti

# Operation notation

Operations show the service a class offers—not the Java method body.

### General form

visibility  name(parameter : Type) : ReturnType

+ hasSeat() : boolean

+ enroll(s : Student) : void

+ findSection(code : String) : CourseSection

- validateCapacity() : boolean

### Important distinctions

Parameter syntax uses name : Type.

Return type appears after the closing parenthesis.

Constructors may be shown, but often only when useful to the design discussion.

No braces. No method body. UML communicates interface and responsibility.


© Muhammad Shahid Bhatti

# One Java class → one UML class box

Use the code to verify the notation, but do not turn the diagram into a screenshot of source code.

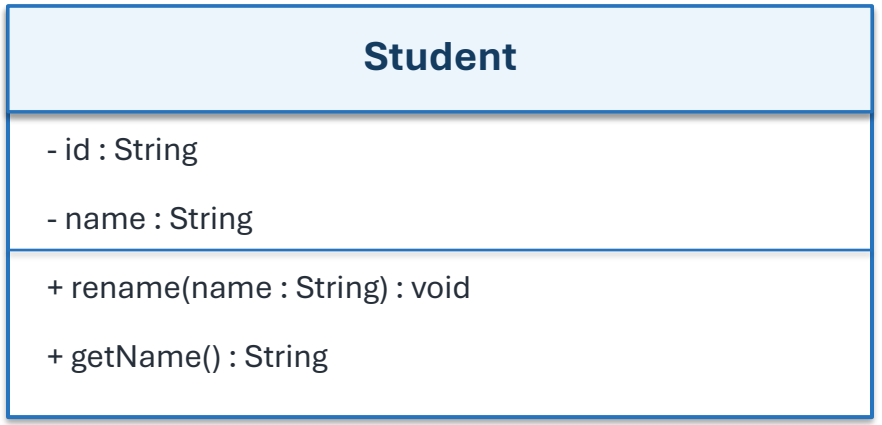
STUDENT.JAVA

```
class Student {
    private String id;
    private String name;

    public void rename(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

→




Same design information, much less visual noise.


© Muhammad Shahid Bhatti

LIVE CODING

# Live coding checkpoint — code and diagram side by side

Type a small class, compile it, then sketch its UML box from the source.

## Repository location

00_Course_Master\01_Lectures\Week06\L11\LiveCoding\01_ClassToUML

## Sequence

1. Create Course.java
2. Add 2 private fields
3. Add 2 public methods
4. Compile with javac
5. Draw UML box
6. Compare code vs. diagram

## Ask students

What details did we omit from UML?

Which access symbols appear?

Does the diagram reveal behavior more quickly than the source?

Which version is executable?

**Teaching rhythm: predict → code → compile → sketch → explain.**


© Muhammad Shahid Bhatti

# Five relationship notations you must distinguish

The line style and endpoint symbol carry meaning.

**Association**

knows / relates to



solid line

**Aggregation**

shared whole–part



hollow diamond

**Composition**

strong whole–part



filled diamond

**Generalization**

is-a / inheritance



open triangle

**Dependency**

temporary uses-a



dashed arrow

Do not choose an arrow because it “looks right.” Choose it because the semantics are right.


© Muhammad Shahid Bhatti

# Association — a structural relationship between classes

Use a solid line when instances of one class have a meaningful relationship with instances of another.



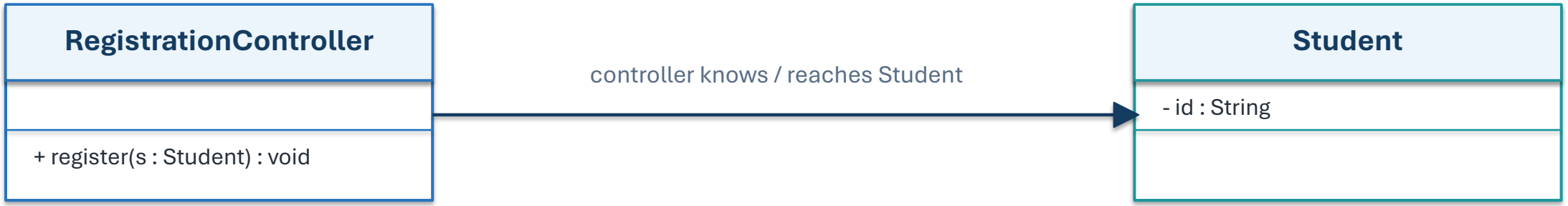

### What the line means

Student objects can be related to CourseSection objects in the domain. The precise number of links is expressed with multiplicity.


© Muhammad Shahid Bhatti

# Navigability — which direction needs knowledge?

Navigability can show that one class can reach the other; omit it when the direction is intentionally unspecified.




**Design question**

Does Student also need a reference back to RegistrationController? If not, bidirectional navigation adds coupling without benefit.


© Muhammad Shahid Bhatti

# Multiplicity — how many objects may participate?

Multiplicity is written near each end of an association.

**1**
exactly one

**0..1**
zero or one

*****
many / unspecified upper bound

**0..***
zero or many

**1..***
one or many

**2..5**
a bounded range

**Multiplicity answers a domain question, not a Java syntax question.**


© Muhammad Shahid Bhatti

READ THE ENDS

# Multiplicity examples in our university domain

Read the multiplicity at the opposite end as “how many of those can one of these be linked to?”




One Course may have zero or many CourseSections; each CourseSection belongs to exactly one Course.

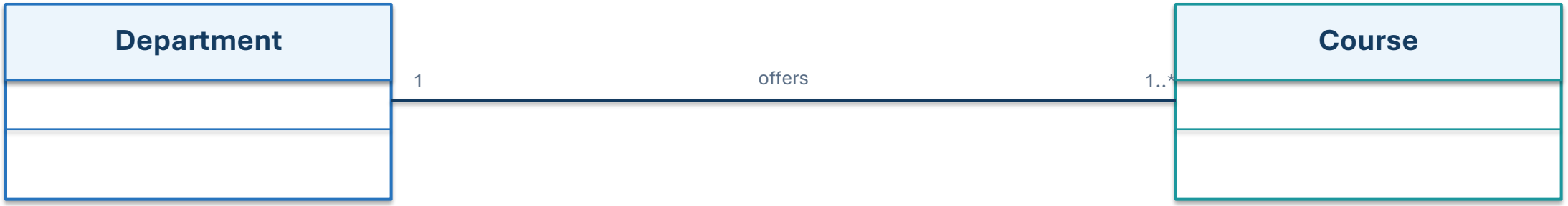
A Student may have zero or many Enrollment records; each Enrollment belongs to one Student.


© Muhammad Shahid Bhatti

ACTIVE PRACTICE

# Read multiplicity from both directions

Students often read only left-to-right. Force yourself to verbalize both directions.




### Direction A

One Department offers one or more Courses.

### Direction B

Each Course belongs to exactly one Department.

### Check

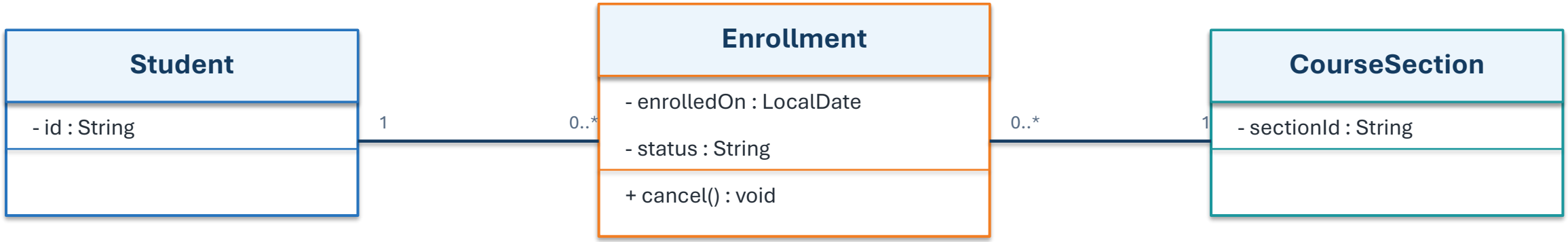
Would 0..* be more realistic before a new department launches its first course?


© Muhammad Shahid Bhatti

ASSOCIATIVE CONCEPT

# When the relationship itself has data, model it as a class

Enrollment is not “just a line” because it owns status, date, grade mode, etc.

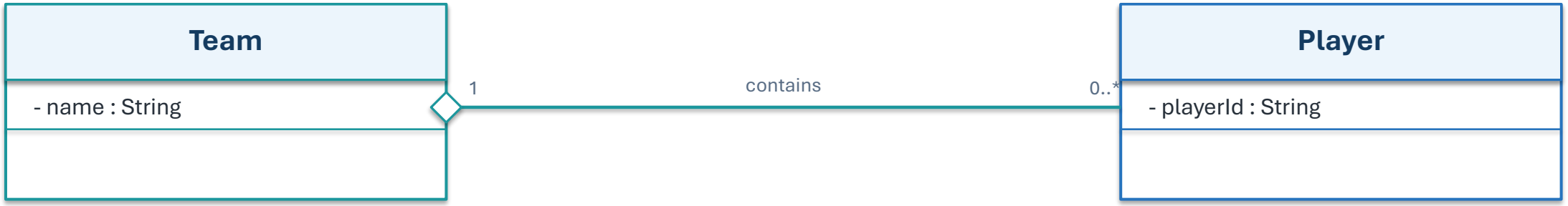



**Enrollment gives the Student–CourseSection relationship its own identity and responsibilities.**


© Muhammad Shahid Bhatti

# Aggregation — a weak whole–part relationship

A hollow diamond is placed at the “whole” end when parts can meaningfully exist independently and may be shared.




**Interpretation**

The Team groups Player objects, but Player objects are not conceptually destroyed merely because the Team object disappears. The parts have independent identity.


© Muhammad Shahid Bhatti

# Composition — a strong whole–part relationship

A filled diamond marks strong ownership/lifetime dependence at the whole end.




**Interpretation**

OrderLine is modeled as a part of one Order. Its meaning and lifetime are strongly tied to that Order in this design.

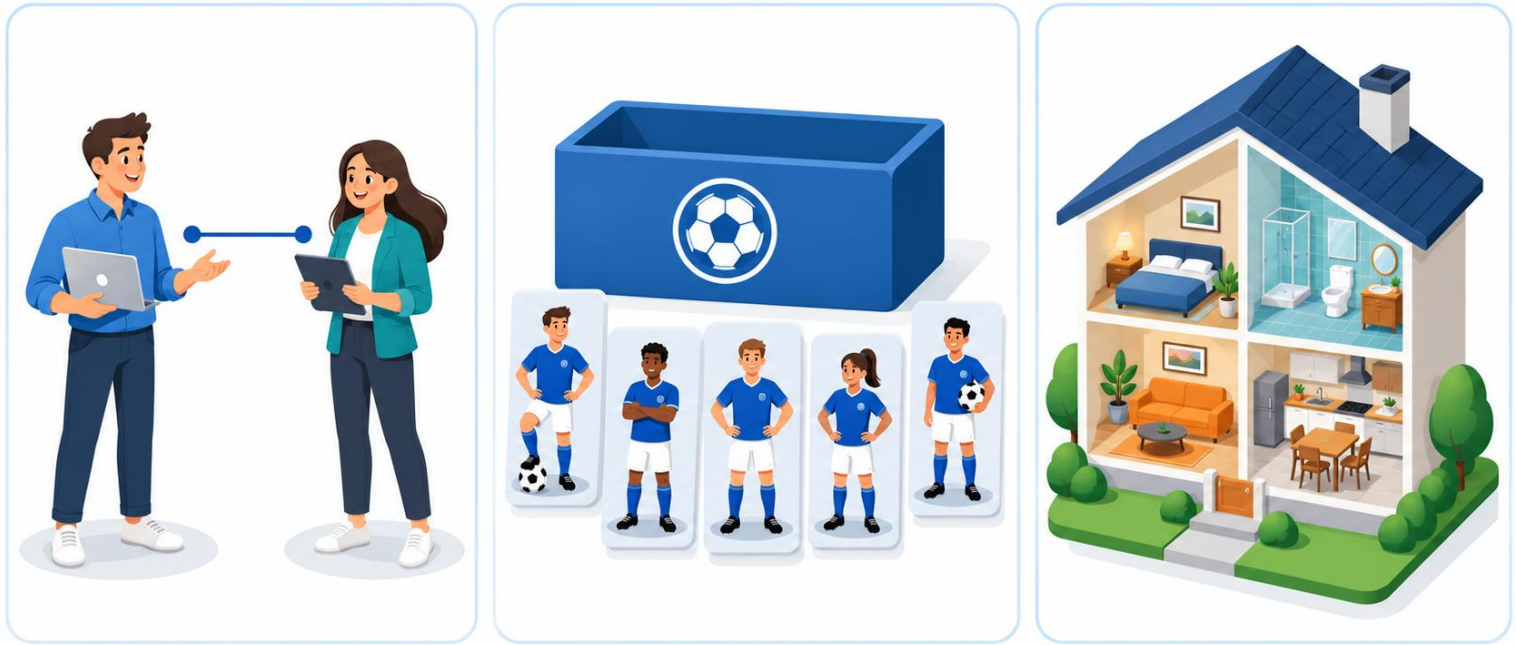

© Muhammad Shahid Bhatti

VISUAL

# Visual intuition — association, aggregation, composition

Pictures are only metaphors; the UML symbols still carry the actual semantics.

**ASSOCIATION** | **AGGREGATION** | **COMPOSITION**



Independent collaboration → shared whole–part → strong owned whole–part


© Muhammad Shahid Bhatti

DESIGN JUDGMENT

# Association vs. aggregation vs. composition

Choose the weakest relationship that truthfully expresses the domain semantics.

## Association

**No whole–part claim**

Independent objects relate

Notation: solid line

Question: does the “whole” truly own the lifecycle of the part?

## Aggregation

**Weak/shared whole–part**

Part survives whole

Notation: hollow diamond

Question: does the “whole” truly own the lifecycle of the part?

## Composition

**Strong owned whole–part**

Part lifetime tied to whole

Notation: filled diamond

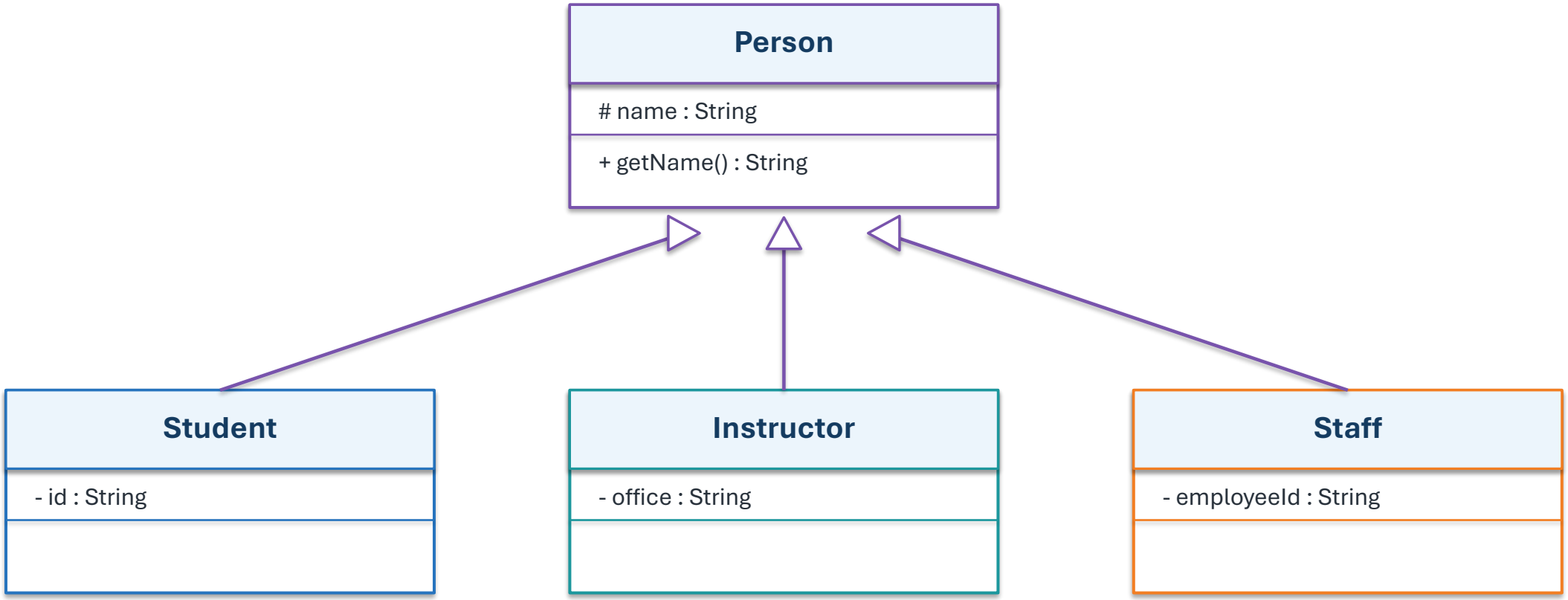
Question: does the “whole” truly own the lifecycle of the part?

**Do not use aggregation or composition merely to make the diagram “more UML-looking.”**


© Muhammad Shahid Bhatti

# Generalization — UML inheritance notation

A solid line with a hollow triangle points to the more general superclass.



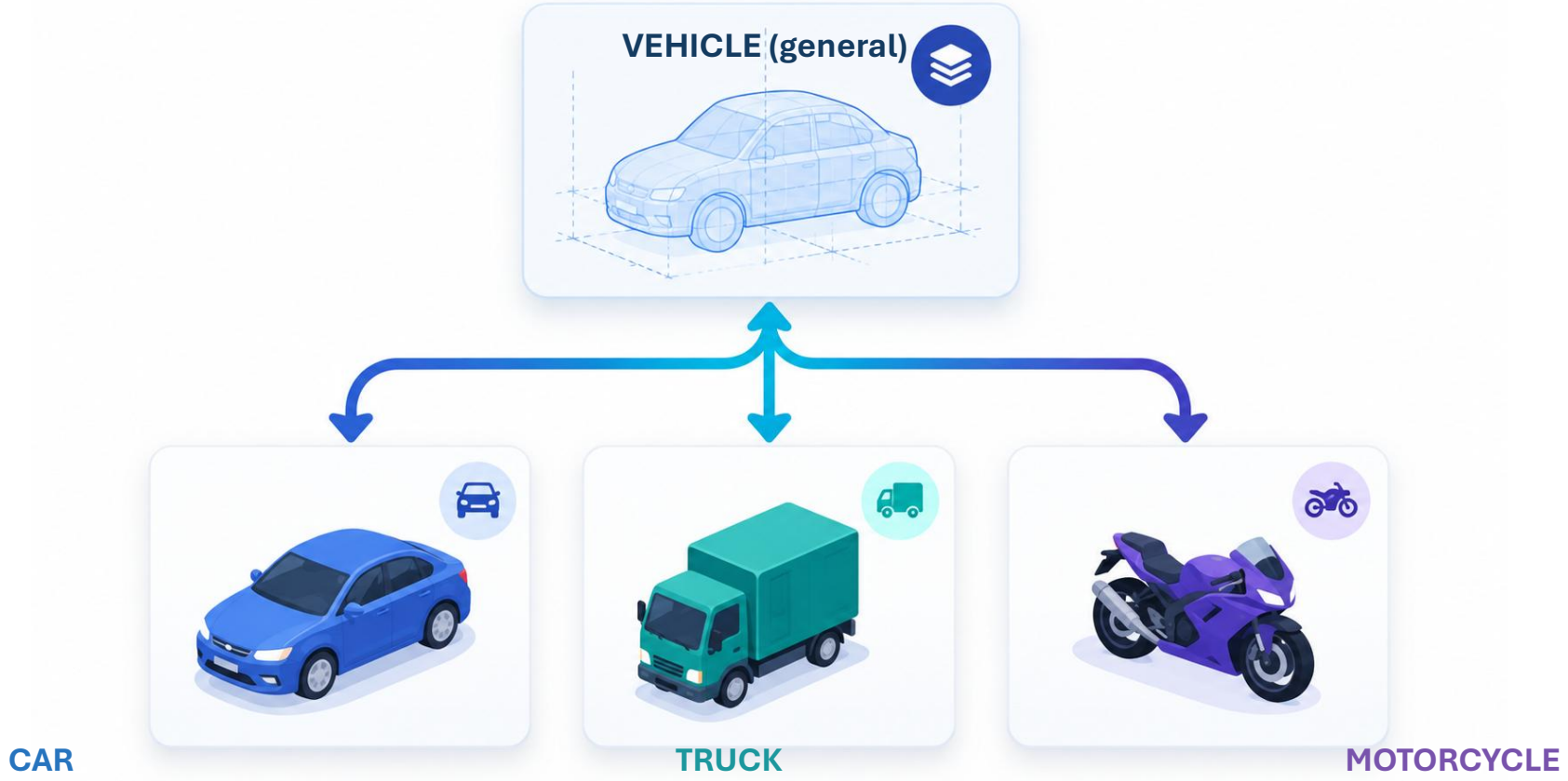

Read upward: “Student is a Person”, “Instructor is a Person”.


© Muhammad Shahid Bhatti

VISUAL

# Visual intuition — one general concept, several specialized forms

The superclass captures common structure; subclasses add or refine specialized behavior

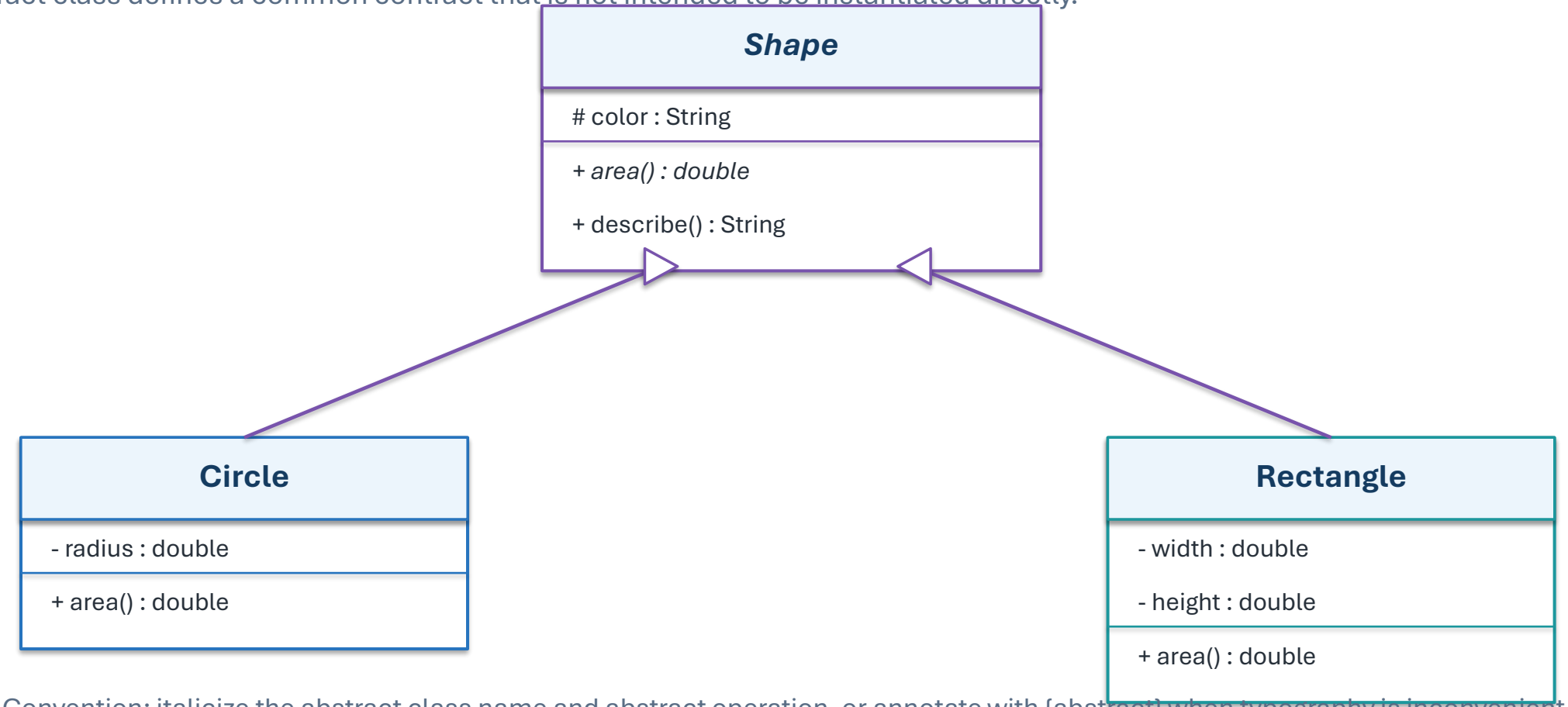



In UML, the hollow triangle points toward the generalized type.


© Muhammad Shahid Bhatti

# Abstract classes and abstract operations in UML

An abstract class defines a common contract that is not intended to be instantiated directly.




Convention: italicize the abstract class name and abstract operation, or annotate with {abstract} when typography is inconvenient.


© Muhammad Shahid Bhatti

# Java abstract class → UML

Now the UML notation should make the code structure obvious at a glance.

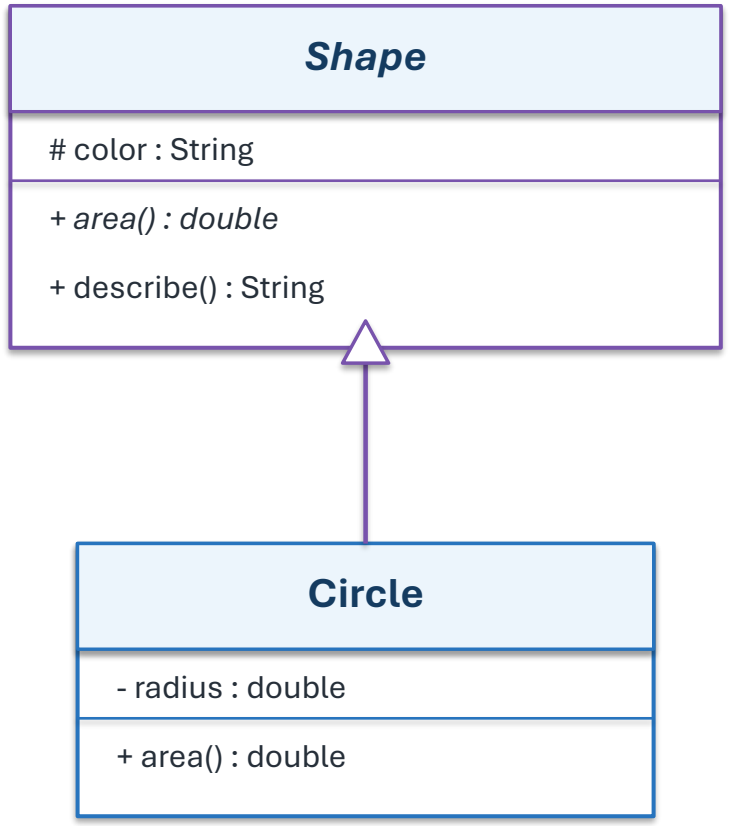
JAVA

```
abstract class Shape {
    protected String color;

    public abstract double area();

    public String describe() {
        return color;
    }
}

class Circle extends Shape {
    private double radius;
    public double area() { return Math.PI *
radius * radius; }
}
```

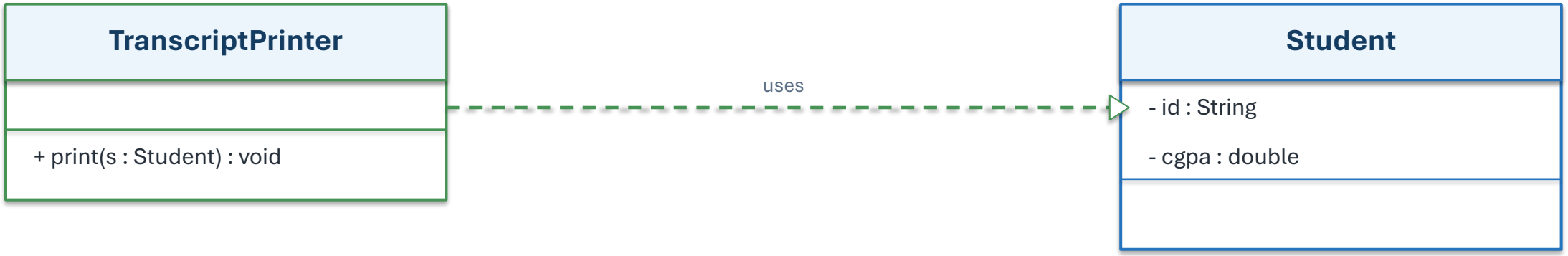




© Muhammad Shahid Bhatti

# Dependency — a temporary “uses-a” relationship

Use a dashed arrow when one class depends on another temporarily without keeping it as persistent state.




**Why dependency?**

TranscriptPrinter needs a Student object only while print(...) executes. It does not necessarily store Student as a field after the call returns.


© Muhammad Shahid Bhatti

# When should a temporary uses-a relationship be modeled?

The key question is whether the relationship is persistent structure or temporary use.

### Method parameter

**print(Student s)**

Dependency is often enough.

### Local variable

**Room room = finder.find(...);**

Temporary use → dependency.

### Static utility call

**Validator.check(student)**

Dependency.

### Field/reference

**private Student student;**

Persistent structure → association.

Not every method call deserves a line on the diagram. Model dependencies that matter to the design discussion.


© Muhammad Shahid Bhatti

CODE READING

# Dependency vs. association — compare the code

The difference is not “one uses an object and the other does not.” Both use objects; persistence is the key.

DEPENDENCY

```
class ReportService {
    void print(Student s) {
        System.out.println(s.getName());
    }
}
```

ASSOCIATION

```
class Advisor {
    private Student student;

    Advisor(Student student) {
        this.student = student;
    }
}
```

**Temporary use**

Student reference exists only during print(...).

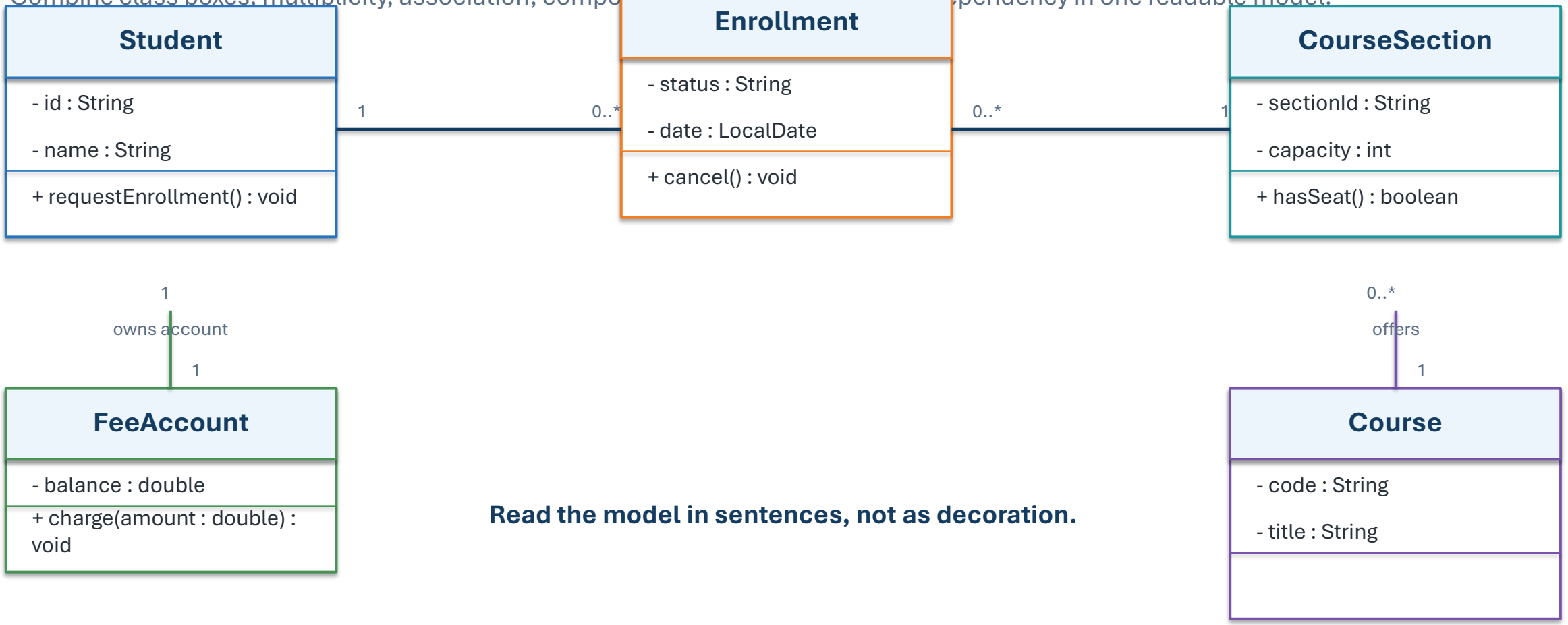
**Persistent link**

Advisor retains the Student reference as object state.


© Muhammad Shahid Bhatti

# Integrated example — university registration class diagram

Combine class boxes, multiplicity, association, compo ependency in one readable model.




**Read the model in sentences, not as decoration.**


© Muhammad Shahid Bhatti

ACTIVE PRACTICE

# Read the integrated diagram before changing it

A good diagram should support questions about structure without opening source code.

**Q1**

Can one Student have many Enrollment records?

**Q2**

Can one Enrollment refer to two CourseSections?

**Q3**

Can a Course exist with zero current sections?

**Q4**

Is FeeAccount modeled as one-to-one with Student?

**Q5**

Which relationship would you change if one Student can own multiple fee accounts?

**Q6**

Where would you add an abstract Person superclass if later required?


© Muhammad Shahid Bhatti

MISCONCEPTIONS

# Common UML class-diagram mistakes

Correct notation matters because small symbol errors can reverse the intended meaning.

### Diamond on the part end

The diamond belongs at the whole/owner end.

### Multiplicity placed without reading both directions

Every endpoint number has a precise interpretation.

### Generalization triangle points to subclass

It must point to the superclass/generalized type.

### Dependency used where a field is stored

Persistent references are structural associations.

### Every method call becomes a dependency line

Show dependencies that matter to the model.

### Class diagram contains specific object values

That belongs to an object/instance view.


© Muhammad Shahid Bhatti

TEAM ACTIVITY

# In-class design activity — library lending

Build a class diagram from the short problem statement.

## Problem statement

A library member borrows physical book copies. Each book title may have several copies. A loan links one member to one copy and records due date and return status. A copy can have at most one active loan at a time. A notification service may send reminders using a Loan object but does not store the member permanently.

## Produce

1. Candidate classes
2. Attributes + operations
3. Multiplicity
4. Association / aggregation / composition choices
5. One dependency
6. Explain one design decision

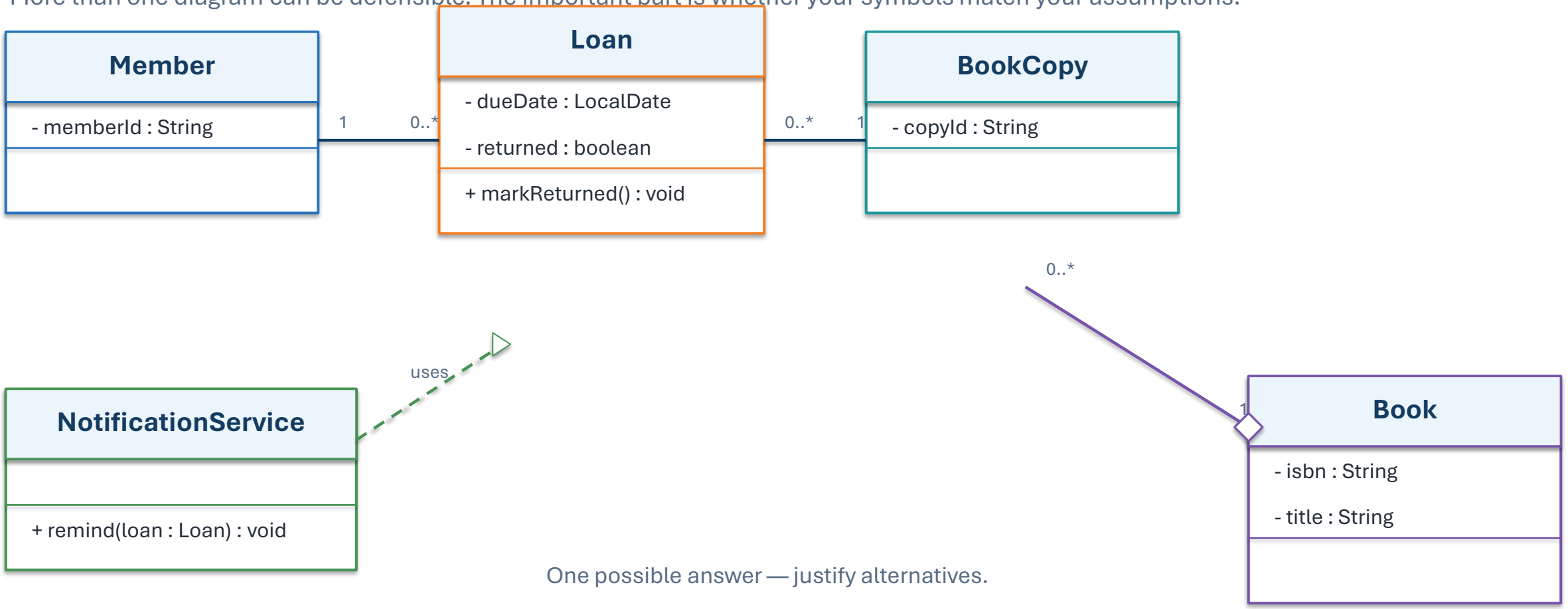
**Constraint: use at most six classes. The goal is clarity, not diagram size.**


© Muhammad Shahid Bhatti

MODEL ANSWER

# Debrief — one reasonable library model

More than one diagram can be defensible. The important part is whether your symbols match your assumptions.




One possible answer — justify alternatives.


© Muhammad Shahid Bhatti

LIVE CODING

# Live coding — keep source and UML synchronized

Change the code, recompile, then decide whether the diagram must change.

## Round 1 — field

Add: private Course course;

Question: dependency or association?

Update UML with a structural line.

## Round 2 — method parameter

Add: print(Student s)

Question: persistent or temporary?

Show dependency only if it matters to the model.

## Round 3 — superclass

Create abstract Person

Make Student extend Person

Add open-triangle generalization.

**This is the bridge to Lecture 12: systematic forward and reverse engineering.**


© Muhammad Shahid Bhatti

RECAP

# Recap + exit ticket

The goal is not memorizing arrows; it is using notation to communicate design meaning correctly.

## Remember

Class box = name + attributes + operations.

+ / - / # / ~ communicate visibility.

Multiplicity belongs at relationship ends.

Hollow diamond = aggregation; filled diamond = composition.

Hollow triangle points to superclass.

Dashed arrow = dependency / temporary use.

Abstract classes and operations are commonly italicized.

## Exit ticket

1. Draw a class with one private field and one public method.
2. Explain 1 vs. 0..* at both ends of an association.
3. When would you choose dependency instead of association?
4. Which end receives the composition diamond?
5. Draw an abstract superclass with two concrete subclasses.

**Next: Lecture 12 — Forward & Reverse Engineering: UML → Java and Java → UML.**


© Muhammad Shahid Bhatti